

RAPORT TEHNIC COMPARATIV

Partea tehnică comună

dintre conspectul TrainingIT și lucrarea de licență

Identificare • sistematizare • clarificare • trasabilitate pe pagini

Documentul 1: TrainingIT_conspectare.docx

Documentul 2: newfile4.pdf

31 august 2026

1. Concluzia comparării

Verdict: Cea mai mare parte a nucleului tehnic din conspect se regăsește explicit în lucrarea de licență. Potrivirea este foarte puternică pentru arhitectura full-stack, stackul tehnologic, separarea pe straturi, șabloanele de proiectare, persistență, modelul de date și fluxul de cumpărare/înscrisere.

Documentul 1 este o reformulare didactică și mai accesibilă a unor idei tehnice din documentul 2. El comprimă explicațiile și, în anumite locuri, generalizează implementarea. De aceea, potrivirea trebuie citită pe trei niveluri: directă, contextuală și parțială cu precizări.

Cum se interpretează nivelurile

Nivel	Semnificație	Cum trebuie folosit
DIRECTĂ	Aceeași tehnologie, structură sau funcție este menționată explicit.	Poate fi prezentată ca parte tehnică confirmată.
CONTEXTUALĂ	Ideea este aceeași, dar lucrarea a doua o descrie mai larg sau cu altă terminologie.	Se păstrează ideea și se adaugă explicația din lucrarea a doua.
PARȚIALĂ	Există un nucleu comun, dar și o limită ori o diferență importantă.	Se folosește numai împreună cu precizarea indicată.

2. Matrice rapidă de corespondență

Tabelul următor arată, dintr-o privire, ce elemente tehnice din primul document sunt susținute de al doilea document.

Temă tehnică	Nivel	Repere verificate
Arhitectură full-stack și REST/JSON	DIRECTĂ	D1: p. 1, 5-7, 15 D2: p. 1-2, 7-9, 26-28, 33, 46-47
Stack tehnologic	DIRECTĂ	D1: p. 1, 9-10 D2: p. 1-2, 8, 26, 33, 46
Straturi backend și CrmFacade	DIRECTĂ	D1: p. 5-7 D2: p. 26-28, 41-44
MariaDB, JDBC, HikariCP, fără ORM	DIRECTĂ	D1: p. 7, 9-10 D2: p. 1-2, 8, 26-32, 42-43, 46-48
Șabloane de proiectare	DIRECTĂ	D1: p. 6-7, 10-13 D2: p. 7, 27-28, 38, 41-45, 47
EventBus / Observer și reacții multiple	DIRECTĂ	D1: p. 7, 10-11, 15 D2: p. 27-28, 38, 43-45, 47
Model de date comun	DIRECTĂ	D1: p. 10-12 D2: p. 29-31, 38-44

Temă tehnică	Nivel	Repere verificate
Flux cumpărare / înscriere	DIRECTĂ	D1: p. 3, 10-12, 14-15 D2: p. 18, 20, 38-45
Frontend, SSE, temă și traducere	DIRECTĂ	D1: p. 6, 8-9, 12 D2: p. 18, 33-37, 43, 46-47
AI opțional și degradare controlată	DIRECTĂ	D1: p. 1, 9, 12, 15 D2: p. 8, 18-22, 32, 36, 47
Portaluri și funcționalități	CONTEXTUALĂ	D1: p. 7-9 D2: p. 18-25, 34-35
Autentificare și control acces	PARTIALĂ	D1: p. 7, 9 D2: p. 18-20, 32, 35, 46-48
Plăți și notificări externe	PARTIALĂ	D1: p. 3, 8-11 D2: p. 18, 21-22, 38-45, 47-48

3. Arhitectura generală și traseul datelor

Repere: documentul 1 - p. 1, 5-7, 9, 15; documentul 2 - p. 1-2, 7-9, 26-28, 33, 41-47.

Ambele documente descriu aceeași soluție full-stack: o interfață Next.js separată de un backend Java/Spring Boot, cu schimb de date printr-un contract REST/JSON. Backendul păstrează logica de afaceri, schema relațională și accesul la date; frontendul gestionează experiența utilizatorului, rutarea și starea locală.

Flux tehnic, în ordine

- 1. Frontend** Next.js 16 + React 19 + TypeScript 5 + Tailwind CSS v4
- 2. Contract** cereri HTTP prin REST/JSON; actualizări live prin SSE
- 3. Strat web** controller Spring Boot subțire, fără logică principală de afaceri
- 4. Domeniu** CrmFacade → servicii → comenzi / repository-uri
- 5. Persistență** DAO → JDBC → HikariCP → MariaDB
- 6. Reacții laterale** EventBus / Observer pentru audit, notificări și actualizări de stare
- 7. Serviciu extern** Claude API, activat numai când există configurarea necesară

Clarificare importantă: Conspectul vorbește despre «o singură aplicație». Lucrarea de licență precizează mai exact că produsul este format din două subsisteme autonome - frontend și backend - care aparțin aceleiași soluții și comunică strict prin REST/JSON. Așadar, este un singur produs funcțional, nu un monolit tehnic.

4. Tehnologiile confirmate în ambele documente

Repere: documentul 1 - p. 1, 7, 9-12; documentul 2 - p. 1-2, 8-9, 26, 32-37, 46.

Zonă	Tehnologie comună	Clarificare
Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS v4	Confirmare explicită în ambele documente.
Backend	Java + Spring Boot 3.5	Java este confirmat; versiunea Java 17 apare în conspect, dar nu este precizată explicit în PDF.
Comunicare	REST/JSON și Server-Sent Events (SSE)	REST pentru cereri/răspunsuri; SSE pentru actualizarea statisticilor publice.
Persistență	MariaDB, JDBC nativ, HikariCP	SQL explicit, pool de conexiuni și schemă inițializată de aplicație.
AI	SDK Anthropic / Claude	Asistent, recomandări și traducere; modul opțional.
Documente	Apache POI și OpenPDF	Import/export Excel și generare/export PDF.

Precizie: Versiunea Java 17, Lombok, SLF4J/Logback și Spring Validation apar în conspect, dar nu sunt confirmate explicit în PDF. Din comparație pot fi susținute sigur Java, Spring Boot 3.5, MariaDB, JDBC și HikariCP.

5. Arhitectura backend pe straturi

Repere: documentul 1 - p. 5-7, 10; documentul 2 - p. 26-29, 41-44.

Structura backendului se regăsește aproape identic în cele două documente:

- Controllerul REST primește cererea, o transformă într-un DTO și delegă execuția; nu conține regulile principale ale afacerii.
- CrmFacade reprezintă punctul unic de intrare în domeniu și ascunde complexitatea serviciilor interne.
- Serviciile implementează regulile pentru contacte, cursuri, înscrieri, facturare, analiză și suport.
- Comenzile încapsulează operațiile de modificare a stării.
- Repository-urile oferă operații orientate spre domeniu, iar DAO-urile execută SQL JDBC direct.
- EventBus formează o ramură laterală, decuplată de traseul principal al cererii.

6. Șabloanele de proiectare comune

Repere: documentul 1 - p. 6-7, 10-13, 15-18; documentul 2 - p. 1-2, 7, 27-28, 38, 41-45, 47.

Șablon	Rol confirmat în lucrarea a doua
Facade	CrmFacade este punctul unic de intrare în domeniu și ascunde complexitatea serviciilor.
Singleton	Servicii și componente centrale sunt accesate ca instanțe unice.
Command	Operațiile de modificare sunt încapsulate; apare mai ales în procesele administrative.
Repository + DAO	Repository oferă interfața de domeniu; DAO execută SQL-ul JDBC către MariaDB.
Builder	Construiește obiecte complexe, inclusiv Enrollment.
Observer / EventBus	Evenimentele sunt publicate central și consumate de observatori independenți.
Factory	Construiește notificări pentru canale diferite, precum e-mail și SMS.
Strategy	Alege algoritmul de scorare a leadului în funcție de tipul contactului.
Chain of Responsibility	Validează succesiv e-mailul, telefonul, tipul contactului și consimțământul GDPR.

Lucrarea a doua nu doar enumeră aceste șabloane, ci urmărește activarea lor în același flux de cumpărare. Aceasta confirmă ideea centrală din conspect: arhitectura este construită astfel încât o operație principală să poată declanșa reacții suplimentare fără ca serviciul inițiator să le cunoască în detaliu.

7. Nucleul bazat pe evenimente

Repere: documentul 1 - p. 7, 10-11, 13, 15, 17-18; documentul 2 - p. 27-28, 38, 43-45, 47.

Potrivirea este directă. După crearea unui contact sau a unei înscrieri, serviciul publică un eveniment. EventBus notifică observatorii înregistrați, care pot scrie în jurnalul de audit, calcula ori actualiza scorul unui lead și genera structura unei notificări. Serviciul principal rămâne independent de aceste efecte secundare.

Ce înseamnă simplu: O singură acțiune este salvată o singură dată, după care mai multe componente pot reacționa separat. Aceasta reduce legăturile directe dintre module și face extinderea mai ușoară.

Limită reală: În lucrarea a doua, e-mailul și SMS-ul sunt construite prin Factory, dar metoda finală de trimitere doar înregistrează acțiunea în log. Integrarea reală cu SMTP/Twilio este marcată ca dezvoltare viitoare.

8. Persistența și modelul de date

Repere: documentul 1 - p. 7, 9-12, 15-17; documentul 2 - p. 26-32, 42-43, 46-48.

Persistența

Datele sunt stocate în MariaDB, accesate prin SQL explicit și JDBC, cu conexiuni reutilizate de HikariCP. WebSchemaInitializer rulează schema.sql la pornire. ORM-ul este evitat intenționat pentru control determinist și transparență, cu prețul unui volum mai mare de cod infrastructural.

Entitățile comune

- Course → CourseSession → Enrollment descrie traseul educațional și înscrierea.
- Contact poate reprezenta o persoană individuală sau o companie.
- Opportunity, Activity și Employee descriu zona comercială și CRM.
- MediationSession este legată de Invoice pentru ședințele individuale și facturare.
- Evaluarea și feedbackul sunt păstrate în Enrollment, ceea ce leagă recenzia de o achiziție/înscriere reală.

Clarificare de model: PDF-ul recunoaște că modelul folosește chei externe gestionate manual și denumiri moștenite, precum MediationSession. Acestea sunt limitări documentate, nu motive pentru a considera modelul inexistent.

9. Fluxul comun de cumpărare și înscriere

Repere: documentul 1 - p. 3, 10-12, 14-15, 17-18; documentul 2 - p. 18, 20, 38-45.

Acesta este cel mai bine documentat punct de corespondență. Fluxul tehnic din lucrarea a doua poate fi sistematizat astfel:

1. Utilizatorul apasă butonul de înscriere din CourseCard.
2. Frontendul trimite o cerere POST prin stratul tipizat lib/api.ts.
3. PublicCatalogController deserializează cererea și o transmite către CrmFacade.
4. ReviewService verifică existența și starea cursului.
5. Contactul este căutat după e-mail; dacă lipsește, este creat înainte de Enrollment.
6. Datele contactului trec prin lanțul de validare, iar scorul inițial este calculat prin Strategy.
7. ContactRepository și ContactDao persistă contactul în MariaDB.
8. Sesiunea cursului este identificată, iar EnrollmentService creează Enrollment prin Builder.
9. EnrollmentRepository și EnrollmentDao salvează înscrierea folosind JDBC/HikariCP.

10. EnrollmentCreatedEvent este publicat; observatorii actualizează auditul și construiesc confirmările.
11. Răspunsul HTTP 201 revine către interfață, iar statisticile sunt transmise asincron prin SSE.

Idempotență confirmată: Dacă înscrierea există deja, fluxul returnează înregistrarea existentă și nu creează un duplicat. Această confirmare privește în mod explicit Enrollment; afirmația din conspect despre idempotența facturii pentru ședințe nu este demonstrată la fel de clar în PDF.

10. Frontendul și actualizarea în timp real

Repere: documentul 1 - p. 6, 8-9, 12; documentul 2 - p. 18-20, 33-37, 43, 46-47.

- Next.js App Router: directoarele și page.tsx definesc rutele, iar layout.tsx furnizează cadrul comun.
- lib/api.ts centralizează toate cererile către backend și mapează erorile într-un contract comun.
- Interfețele TypeScript oglindesc DTO-urile și entitățile primite de la backend.
- Starea este gestionată prin mecanisme native React, Context, evenimente de fereastră și localStorage, fără Redux/MobX/Zustand.
- SSE folosește EventSource pentru actualizarea numărului de cursuri, cursanți și evaluări fără polling periodic.
- TranslationProvider și MutationObserver susțin traducerea dinamică, iar Tailwind CSS v4 gestionează tema și modul întunecat.

11. Funcționalitățile care se regăsesc

Repere: documentul 1 - p. 7-9; documentul 2 - p. 18-25, 34-35.

Portalul public / utilizator

- catalog public și filtrarea cursurilor;
- înregistrare, autentificare și recuperarea parolei;
- cumpărare/înscriere și pagina My Courses;
- citirea, scrierea și actualizarea recenziilor;
- programarea și vizualizarea ședințelor cu trainerul;
- asistent AI, recomandări, traducere și raportarea unei probleme.

Portalul administrativ / CRM

- dashboard și indicatori; contacte și leaduri;
- administrarea cursurilor, angajaților și înscrierilor;
- facturi și export PDF; pipeline de oportunități și activități;
- analize, rapoarte și gestionarea sesizărilor;
- recomandări și analize AI pentru companii și vânzări.

Limită de corespondență: Detaliile foarte concrete din conspect - programul 08:00-20:00, tariful de 10 dolari/oră, reducerea exactă de 60% sau anumite ecrane administrative - nu sunt confirmate explicit în lucrarea a doua și nu trebuie prezentate ca potriviri directe.

12. Integrarea AI și documentele

Repere: documentul 1 - p. 1, 8-12, 15; documentul 2 - p. 8, 18-22, 25-26, 32, 36, 47.

AI opțional

ClaudeClient este un wrapper peste SDK-ul Anthropic. Configurarea depinde de ANTHROPIC_API_KEY. Dacă cheia lipsește, clientul rămâne neinițializat, isEnabled() întoarce false, iar apelurile produc un răspuns controlat de indisponibilitate, nu o oprire a întregii aplicații. Funcțiile comune sunt asistentul conversațional, recomandările și traducerea dinamică.

Importuri și exporturi

Apache POI este folosit pentru fișiere Excel și importul angajaților, iar OpenPDF pentru facturi și exporturi PDF. Această componentă apare explicit în ambele documente.

13. Autentificarea și securitatea - potrivire cu precizare

Repere: documentul 1 - p. 5, 7, 9, 16-18; documentul 2 - p. 18-20, 32, 35, 46-48.

Este confirmată existența rolurilor USER și ADMIN, a obiectului AuthSession, a AuthGuard și a redirectionării către portalul potrivit. Sesiunea este păstrată în localStorage, iar protecția rutelor este implementată în frontend.

Avertisment obligatoriu: Lucrarea a doua precizează că autentificarea și autorizarea actuală sunt client-side și potrivite pentru demonstrație, nu pentru producție. Nu există încă sesiuni reale sau JWT, parole hashuite, RBAC server-side complet, MFA ori protecție industrială. Prin urmare, formularea corectă este «separare funcțională a rolurilor în interfață», nu «securitate completă».

14. Ce nu trebuie preluat ca fiind confirmat

Pentru a păstra comparația corectă, următoarele elemente din conspect nu au fost incluse în sinteza tehnică principală:

- o strategie completă de testare manuală end-to-end și o suită automată bazată pe spring-boot-starter-test; PDF-ul nu documentează un capitol de testare echivalent;
- Spring Validation, Lombok, SLF4J și Logback ca tehnologii folosite; nu sunt menționate explicit în lucrarea a doua;
- numărul aproximativ de 11.500 de linii Java și aproximativ 50 de module TypeScript/React; nu este confirmat în PDF;
- izolarea erorii de facturare astfel încât rezervarea să rămână salvată; nucleul evenimentelor este confirmat, dar acest comportament precis nu este demonstrat;
- funcționarea reală a SMTP/Twilio și Stripe/Netopia; PDF-ul le tratează ca integrări externe propuse sau încă neconectate;
- detaliile comerciale exacte privind programul trainerilor, tarifele și procente de reducere.

15. Formulare tehnică consolidată

Variantă recomandată: Textul de mai jos reunește numai ideile tehnice care sunt susținute de ambele documente și introduce precizările necesare.

TrainingIT este o soluție full-stack alcătuită din două subsisteme autonome: un frontend single-page realizat cu Next.js 16, React 19, TypeScript 5 și Tailwind CSS v4 și un backend Java bazat pe Spring Boot 3.5. Cele două componente comunică exclusiv printr-un contract REST/JSON, iar actualizările publice în timp real sunt transmise prin Server-Sent Events.

Backendul este organizat pe straturi. Controllerele REST sunt subțiri și delegă cererile către CrmFacade, care reprezintă punctul unic de intrare în domeniu. Serviciile implementează logica de afaceri, comenzile

încapsulează modificările de stare, repository-urile oferă operații orientate spre domeniu, iar DAO-urile execută SQL explicit prin JDBC.

Persistența este realizată în MariaDB, cu un pool de conexiuni HikariCP și o schemă inițializată de aplicație. ORM-ul este evitat intenționat pentru transparență și control determinist, ceea ce implică însă mai mult cod de infrastructură și gestionarea manuală a relațiilor.

Nucleul aplicației utilizează șabloane precum Facade, Singleton, Command, Repository/DAO, Builder, Strategy, Chain of Responsibility, Observer și Factory. EventBus publică evenimente precum ContactCreatedEvent și EnrollmentCreatedEvent, iar observatorii pot actualiza auditul, scorurile și notificările fără ca serviciul principal să cunoască toate efectele secundare.

Fluxul de cumpărare pornește din frontend, trece prin controller, facade și servicii, identifică sau creează contactul înainte de Enrollment, validează datele, calculează scorul, persistă contactul și înscrierea, publică evenimentele și returnează răspunsul către interfață. Operația de înscriere este idempotentă: o înscriere existentă nu este duplicată.

Modelul de date leagă zona educațională - Course, CourseSession și Enrollment - de zona CRM - Contact, Opportunity, Activity și Employee. Ratingul și feedbackul sunt păstrate în Enrollment, ceea ce permite asocierea recenziei cu o înscriere reală.

Integrarea Claude este opțională și deservește asistentul, recomandările și traducerea. Lipsa cheii API produce degradare controlată, fără oprirea aplicației. Totuși, autentificarea actuală rămâne client-side, iar notificările și plățile externe nu sunt încă integrate complet; acestea reprezintă direcții de dezvoltare pentru o versiune de producție.

16. Concluzie finală

Partea tehnică din primul document care se regăsește solid în al doilea document este formată din: arhitectura full-stack decuplată, stackul Next.js/Spring Boot/MariaDB, comunicarea REST și SSE, backendul stratificat, persistența JDBC fără ORM, șabloanele de proiectare, EventBus și Observer, modelul de date CRM-educațional, fluxul de cumpărare/înscriere, integrarea AI opțională și funcțiile principale ale celor două portaluri.

Ideea centrală: TrainingIT este un singur produs funcțional construit din două aplicații tehnic independente, legate printr-un contract clar și susținute de un domeniu Java, o bază de date relațională comună și mecanisme decuplate pentru reacțiile secundare.